

上海财经大学

信息管理与工程学院

近端梯度法与 FISTA

优化理论与算法 后续专题 复合优化的一阶方法

王震

2026

课程导入：为什么要拆成 $f + g$

上一阶段主要讨论光滑优化。很多数据问题还会额外要求“结构”，例如稀疏、非负、预算约束或低秩。

光滑部分 f

- ▶ 衡量误差或损失；
- ▶ 梯度容易计算；

结构部分 g

- ▶ 表达稀疏、约束或先验；
- ▶ 可能不可导；

本节课要解决的问题

当目标函数同时包含“好求梯度的损失”和“不好求梯度的结构”时，如何设计稳定、可实现的一阶算法？

本讲主线

本讲围绕凸复合优化问题展开：

$$\min_x F(x) = f(x) + g(x),$$

其中

- ▶ f ：光滑可导，例如平方损失；
- ▶ g ：可能不可导，但结构简单，例如 ℓ_1 正则或约束指示函数。

核心问题

当 g 不可导时，普通梯度下降无法直接对 $F = f + g$ 求梯度。近端梯度法的思路是：让 f 负责“下降”，让 g 负责“修正结构”。

本讲学习目标

- ▶ 知道为什么普通 GD 难以直接处理不可导项；
- ▶ 理解近端算子 $\text{prox}_{\alpha g}$ 的含义；
- ▶ 掌握近端梯度法的两步处理过程；
- ▶ 理解 ℓ_1 正则对应的软阈值操作；
- ▶ 知道投影梯度法为什么是近端梯度法的特例；
- ▶ 理解 FISTA 如何用外推加速近端梯度法。

一句话

近端梯度法 = 对光滑部分做梯度下降，对结构部分做近端修正；FISTA = 在此基础上加入外推加速。

为什么需要新的方法

如果目标函数是

$$F(x) = f(x) + g(x),$$

并且 g 不可导，那么普通 GD 的更新

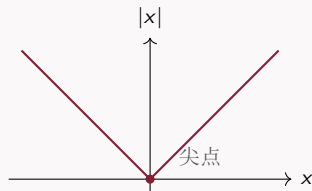
$$x^{k+1} = x^k - \alpha \nabla F(x^k)$$

可能无法直接写出，因为 $\nabla g(x)$ 不存在。

例子：绝对值

$$g(x) = |x|$$

在 $x = 0$ 处有尖点，不可导。



但 g 往往有重要作用：例如产生稀疏性、表达约束、编码先验结构。

典型例子: Lasso

以 Lasso 回归为例:

$$\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1.$$

这里

$$f(x) = \frac{1}{2} \|Ax - b\|^2, \quad g(x) = \lambda \|x\|_1.$$

光滑项

$$\nabla f(x) = A^T(Ax - b).$$

可以用梯度下降处理。

不可导项

$$\|x\|_1 = \sum_i |x_i|.$$

在 $x_i = 0$ 处不可导, 但能把不重要变量的系数压成 0。

课堂互动：这些问题如何拆成 $f + g$

请先判断：下面每个问题中，哪一部分适合放进 f ，哪一部分适合放进 g ？

应用问题	光滑损失 f	结构项 g
从多个指标中预测销售额	预测误差	变量选择，希望模型简单
训练带非负权重的模型	损失函数	权重不能为负
图片去噪	与观测图片的差距	希望图片边缘清晰、区域平滑
推荐系统补全评分矩阵	已知评分的拟合误差	希望矩阵低秩

互动问题

如果把“变量选择”或“非负约束”硬塞进梯度项，会遇到什么困难？

更多复合优化例子

问题	$f(x)$	$g(x)$
Lasso	平方损失 $\frac{1}{2} \ Ax - b\ ^2$	$\lambda \ x\ _1$
约束优化	光滑目标函数	指示函数 $\delta_C(x)$
稀疏逻辑回归	逻辑损失	ℓ_1 正则
图像去噪	数据拟合项	总变差或稀疏正则
推荐系统	已知评分拟合项	低秩正则或核范数

共同点

f 通常负责“拟合数据”或“降低误差”； g 负责“引入结构”，例如稀疏、非负、预算、平滑等。

近端算子

给定一个临时点 z ，近端算子定义为

$$\text{prox}_{\alpha g}(z) = \arg \min_x \left\{ g(x) + \frac{1}{2\alpha} \|x - z\|^2 \right\}.$$

它在做一个折中：

- ▶ 希望 $g(x)$ 小；
- ▶ 又希望 x 不要离 z 太远。

直觉

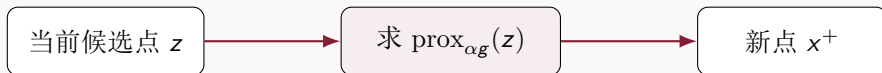
近端算子不是直接沿梯度走，而是问：在不离开 z 太远的前提下，怎样把点调整到更符合 g 所要求的结构？

近端算子的处理图像

近端算子

$$\text{prox}_{\alpha g}(z) = \arg \min_x \left\{ g(x) + \frac{1}{2\alpha} \|x - z\|^2 \right\}$$

可以看成在两个要求之间做平衡。



既希望 $g(x)$ 小
又希望 x 离 z 不远

课堂理解

如果投影是“把点拉回可行域”，那么 prox 更一般：它把点拉向 g 所偏好的结构，例如稀疏、非负或低秩。

近端梯度法公式

对 f 做一阶近似并加二次稳定项：

$$f(x) \approx f(x^k) + \nabla f(x^k)^T (x - x^k) + \frac{1}{2\alpha} \|x - x^k\|^2.$$

再加上 $g(x)$ ，得到更新：

$$x^{k+1} = \text{prox}_{\alpha g} (x^k - \alpha \nabla f(x^k)).$$

两步理解

$$z^k = x^k - \alpha \nabla f(x^k)$$

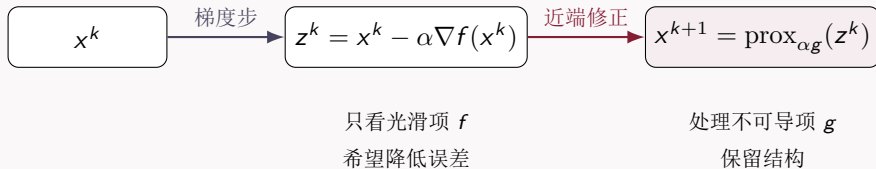
先处理光滑项；再

$$x^{k+1} = \text{prox}_{\alpha g}(z^k)$$

处理不可导项或结构项。

近端梯度法：图示流程

每一步可以分成两个动作：



处理过程

先像 GD 一样走到 z^k ，然后用 prox 把 z^k 修正成满足结构要求的新点。

算法框架

Proximal Gradient Method

1. 选初始点 x^0 , 步长 $\alpha > 0$;
2. 计算光滑项梯度 $g^k = \nabla f(x^k)$;
3. 做梯度步 $z^k = x^k - \alpha g^k$;
4. 做近端修正 $x^{k+1} = \text{prox}_{\alpha g}(z^k)$;
5. 若变化很小或达到最大迭代次数, 则停止。

与投影梯度法非常像, 只是“投影”换成了更一般的“近端算子”。

软阈值： ℓ_1 正则的 **prox**

若

$$g(x) = \lambda \|x\|_1,$$

则近端算子逐坐标可写为

$$\text{prox}_{\alpha\lambda\|\cdot\|_1}(z)_i = S_{\alpha\lambda}(z_i),$$

其中软阈值算子为

$$S_\tau(z) = \text{sign}(z) \max\{|z| - \tau, 0\}.$$

- ▶ 大的正数变小；
- ▶ 大的负数向 0 靠近；
- ▶ 绝对值小于阈值的直接变成 0。

为什么产生稀疏性

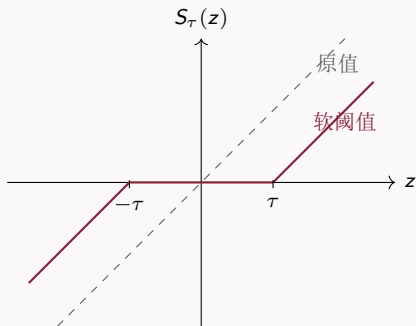
软阈值会把小系数直接压成 0，因此
Lasso 可以自动做变量选择。

软阈值图像

软阈值算子

$$S_{\tau}(z) = \text{sign}(z) \max\{|z| - \tau, 0\}$$

会把靠近 0 的数直接压成 0。



- ▶ $z > \tau$: 向左减去 τ ;
- ▶ $|z| \leq \tau$: 变成 0;
- ▶ $z < -\tau$: 向右加上 τ 。

直觉

小信号被视为噪声或不重要变量，直接清零。

软阈值手算例子

设阈值为

$$\tau = \alpha\lambda = 0.4.$$

对向量

$$\mathbf{z} = (1.2, 0.3, -0.2, -1.5, 0.8)$$

逐坐标做软阈值：

z_i	1.2	0.3	-0.2	-1.5	0.8
$S_{0.4}(z_i)$	0.8	0	0	-1.1	0.4

处理过程

先由梯度步得到临时向量 $\mathbf{z}$ ，再用软阈值逐坐标处理：小的归零，大的保留但向 0 收缩。

课堂互动：哪些变量会被保留下来

假设一次梯度步之后得到

$$\mathbf{z} = (0.9, -0.15, 0.05, 1.8, -0.7), \quad \tau = \alpha\lambda = 0.3.$$

请同学们先判断哪些坐标会变成 0，再计算软阈值结果。

z_i	0.9	-0.15	0.05	1.8	-0.7
是否保留	保留	清零	清零	保留	保留
$S_{0.3}(z_i)$	0.6	0	0	1.5	-0.4

课堂追问

如果把 λ 调大，模型会保留更多变量还是更少变量？为什么？

常见近端算子（一）：逐坐标型

除了 ℓ_1 软阈值，很多常见结构都有简单 prox 。

$g(x)$	$\text{prox}_{\alpha g}(z)$	作用
$\lambda \ x\ _1$	$S_{\alpha\lambda}(z_i)$	软阈值，产生稀疏性
$\frac{\lambda}{2} \ x\ _2^2$	$\frac{1}{1+\alpha\lambda} z$	整体缩小，类似 ridge 正则
$\delta_{\{x \geq 0\}}(x)$	$\max\{z, 0\}$	投影到非负正交象限
$\delta_{[l, u]}(x)$	$\min\{u, \max\{l, z\}\}$	投影到盒约束

这里的 $\max$ 、 $\min$ 都是逐坐标计算。

常见近端算子（二）：结构型

有些 prox 不是逐坐标截断，但也有清晰的结构解释。

$g(x)$	近端算子	典型用途
$\lambda \ x\ _2$	$\left(1 - \frac{\alpha\lambda}{\ z\ _2}\right)_+ z$	向量整体收缩，可用于组选择
$\lambda \sum_G \ x_G\ _2$	每一组做 ℓ_2 软阈值	Group Lasso，整组变量选择
$\delta_C(x)$	$\Pi_C(z)$	约束优化，投影回可行域
$\lambda \ X\ _*$	对奇异值做软阈值	低秩矩阵恢复，推荐系统

课堂取舍

本科课堂重点掌握前三类： ℓ_1 软阈值、平方 ℓ_2 缩放、约束指示函数对应投影。组稀疏和核范数可以作为拓展例子。

例子：平方 ℓ_2 正则的 **prox**

若

$$g(x) = \frac{\lambda}{2} \|x\|_2^2,$$

则

$$\text{prox}_{\alpha g}(z) = \arg \min_x \left\{ \frac{\lambda}{2} \|x\|^2 + \frac{1}{2\alpha} \|x - z\|^2 \right\}.$$

对目标求导并令其为零：

$$\lambda x + \frac{1}{\alpha}(x - z) = 0.$$

因此

$$\text{prox}_{\alpha g}(z) = \frac{1}{1 + \alpha\lambda} z.$$

直觉

ℓ_2^2 正则不会把系数压成 0，而是把整个向量平滑地缩小。

约束也是一种近端项

若

$$g(x) = \delta_C(x) = \begin{cases} 0, & x \in C, \\ +\infty, & x \notin C, \end{cases}$$

则

$$\text{prox}_{\alpha g}(z) = \Pi_C(z).$$

统一视角

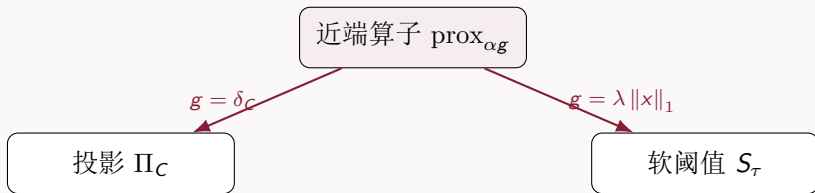
投影梯度法是近端梯度法的特例：

$$x^{k+1} = \Pi_C(x^k - \alpha \nabla f(x^k)).$$

也就是说，投影可以看作对约束指示函数做 prox 。

prox 与投影的关系图

投影和软阈值看起来不同，但都属于 prox。



统一理解

近端算子根据 g 的类型做不同修正：约束对应投影， ℓ_1 正则对应软阈值。

ISTA: Lasso 的近端梯度法

对 Lasso

$$\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1,$$

近端梯度法给出

$$x^{k+1} = S_{\alpha\lambda} (x^k - \alpha A^T (Ax^k - b)).$$

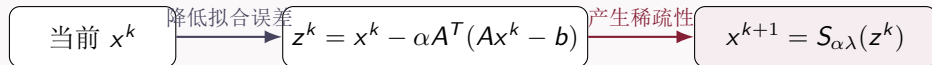
这个算法常称为 ISTA。

一步中发生了什么

先用平方损失的梯度改进拟合误差，再用软阈值把不重要的变量压小或压成 0。

ISTA 的一步处理过程

以 Lasso 为例，一步 ISTA 可以拆成：



梯度步

根据残差 $Ax^k - b$ 调整系数，让预测更接近数据。

软阈值

把较小系数压成 0，得到更简洁、更容易解释的模型。

课堂案例：稀疏回归变量选择

假设我们用多个变量预测销售额：

$$b \approx Ax.$$

变量可能包括价格、广告、节假日、天气、竞品价格、库存等。

普通平方损失

可能给很多变量都分配非零系数，模型解释较复杂。

$$\min_x \frac{1}{2} \|Ax - b\|^2$$

Lasso

用 ℓ_1 正则让不重要变量系数变成 0。

$$\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$$

近端梯度法中的软阈值步骤，就是在每次迭代中执行“变量筛选”。

停止准则：近端梯度映射

类似投影梯度法，常定义近端梯度映射

$$G_{\alpha}(x) = \frac{1}{\alpha} (x - \text{prox}_{\alpha g}(x - \alpha \nabla f(x))) .$$

若 $\|G_{\alpha}(x)\|$ 很大

说明梯度步加近端修正后，点还会明显变化，尚未稳定。

若 $\|G_{\alpha}(x)\|$ 接近 0

说明再做一次近端梯度更新也几乎不动，可以作为停止信号。

步长怎么选

若 f 是 L -光滑，常用固定步长

$$0 < \alpha \leq \frac{1}{L}.$$

对 Lasso 中的

$$f(x) = \frac{1}{2} \|Ax - b\|^2,$$

有

$$\nabla f(x) = A^T(Ax - b), \quad L = \lambda_{\max}(A^T A).$$

课堂提醒

步长太大时，梯度步可能过于激进；步长太小时，虽然稳定但收敛慢。实际中也常用回溯线搜索自动调步长。

收敛速度

若 f 是凸且 L -光滑, g 是凸函数, 取

$$0 < \alpha \leq \frac{1}{L},$$

则近端梯度法满足典型速度:

$$F(x^k) - F(x^*) = \mathcal{O}\left(\frac{1}{k}\right).$$

与 GD 的关系

近端梯度法是 GD 面向复合优化的推广。速度仍是一阶方法的 $\mathcal{O}(1/k)$, 但能处理不可导结构。

常见误区

误区	正确理解
不可导就不能优化	很多不可导结构可以通过 prox 处理
prox 等于投影	投影只是 $g = \delta_C$ 时的特殊 prox
ℓ_1 正则只是让系数变小	软阈值还会把小系数直接压成 0
近端梯度一定容易用	关键取决于 prox_g 是否容易计算

什么时候用近端梯度法

问题结构	是否适合近端梯度
光滑无约束	可用 GD、Newton、BFGS，不一定需要 prox
光滑项 + ℓ_1 正则	非常适合，prox 是软阈值
光滑项 + 简单约束	适合，prox 是投影
复杂不可导项	取决于 prox 是否容易计算

选择原则

如果 g 不可导但 prox_g 容易算，近端梯度法通常是首选方法之一。

从 ISTA 到 FISTA: 为什么还要加速

近端梯度法已经能处理不可导结构，但基础版本的收敛速度通常是

$$F(x^k) - F(x^*) = \mathcal{O}\left(\frac{1}{k}\right).$$

当样本很多、变量很多或精度要求较高时，这个速度可能不够理想。

回顾 ISTA 的一步：

$$x^{k+1} = \text{prox}_{\alpha g}(x^k - \alpha \nabla f(x^k)).$$

自然问题

能否像 Nesterov 加速梯度下降一样，让近端梯度也利用“历史方向”，少走一些弯路？

ISTA 回顾

ISTA 是近端梯度法在 Lasso 等问题上的常用名称:

$$x^{k+1} = \text{prox}_{\alpha g} (x^k - \alpha \nabla f(x^k)).$$

若 f 凸且 L -光滑, g 凸, $\alpha \leq 1/L$, 则

$$F(x^k) - F(x^*) = \mathcal{O}\left(\frac{1}{k}\right).$$

课堂理解

ISTA 的优点是稳定、简单、容易实现; 它的不足是只从当前点出发, 没有主动利用前几步已经形成的移动趋势。

从 Nesterov 到 FISTA

Nesterov 加速的思想是：不仅看当前点，还利用前后两步的惯性。

$$y^k = x^k + \beta_k(x^k - x^{k-1}).$$

FISTA 把近端梯度步放在 y^k 上：

$$x^{k+1} = \text{prox}_{\alpha g}(y^k - \alpha \nabla f(y^k)).$$

直观理解

ISTA 是“从当前点走”；FISTA 是“先根据历史趋势预测一个点，再从预测点做近端梯度步”。

课堂互动：外推点会带来什么

考虑一维情形，最近两步为

$$x^{k-1} = 1.0, \quad x^k = 0.6.$$

如果取 $\beta_k = 0.5$ ，则外推点为

$$y^k = x^k + \beta_k(x^k - x^{k-1}) = 0.6 + 0.5(0.6 - 1.0) = 0.4.$$

好处

如果下降方向判断正确，外推点更靠近最优解，后续 `prox` 步可能更有效。

风险

如果外推过头，函数值可能震荡，因此实际算法常配合 `restart`。

请同学们思考：若 $x^k - x^{k-1}$ 的方向已经错了，外推会帮助还是会添乱？

FISTA 算法

给定 $x^0 = x^{-1}$, 令 $t_0 = 1$ 。对 $k = 0, 1, 2, \dots$:

$$y^k = x^k + \frac{t_{k-1} - 1}{t_k}(x^k - x^{k-1}),$$

$$x^{k+1} = \text{prox}_{\alpha g}(y^k - \alpha \nabla f(y^k)),$$

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}.$$

核心变化

和 ISTA 相比, FISTA 多了一个外推点 y^k 和动量参数 t_k 。

FISTA 的理论速度

在凸复合优化问题

$$\min_x f(x) + g(x)$$

中, 若 f 是 L -光滑凸函数, g 是闭凸函数, 取 $\alpha \leq 1/L$, FISTA 满足

$$F(x^k) - F(x^*) = \mathcal{O}\left(\frac{1}{k^2}\right).$$

ISTA

$$\mathcal{O}(1/k)$$

简单稳定。

FISTA

$$\mathcal{O}(1/k^2)$$

理论上更快。

FISTA 用于 Lasso

对 Lasso

$$\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1,$$

FISTA 为

$$\begin{aligned} y^k &= x^k + \beta_k(x^k - x^{k-1}), \\ x^{k+1} &= S_{\alpha\lambda}(y^k - \alpha A^T(Ay^k - b)). \end{aligned}$$

和 **ISTA** 的差别

ISTA 对 x^k 做软阈值梯度步；FISTA 先生成预测点 y^k ，再对 y^k 做软阈值梯度步。

加速可能带来震荡

FISTA 的函数值不一定每一步都下降。原因是外推点 y^k 可能冲得太远。

实践中常用 restart 策略：

若出现不理想的震荡，就重置动量。

例如令

$$t_k = 1, \quad y^k = x^k.$$

课堂口径

加速不是免费午餐：它用更激进的外推换取更快速度，因此可能需要 restart 稳定迭代。

ISTA、FISTA 与 Nesterov 的关系

方法	处理问题	核心思想
GD	光滑无约束	沿负梯度走
Nesterov	光滑无约束	在预测点计算梯度
ISTA	光滑项 + 不可导项	梯度步 + prox
FISTA	光滑项 + 不可导项	Nesterov 外推 + prox

FISTA 可以理解为“近端版 Nesterov 加速”。

什么时候使用 FISTA

- ▶ 目标函数具有 $f + g$ 结构;
- ▶ f 光滑, ∇f 容易计算;
- ▶ g 的 prox 容易计算;
- ▶ 问题凸, 且希望比 ISTA 更快;
- ▶ 可接受一定震荡, 或愿意使用 restart。

典型应用

稀疏回归、压缩感知、图像去噪、稀疏信号恢复、带简单正则项的机器学习模型。

本讲总结

- ▶ 复合优化把目标拆成 $F(x) = f(x) + g(x)$: f 负责光滑损失, g 负责结构要求。
- ▶ 近端梯度法先对 f 做梯度步, 再用 $\text{prox}_{\alpha g}$ 修正结构。
- ▶ ℓ_1 正则的 prox 是软阈值, 能把小系数直接压成 0。
- ▶ 投影梯度法是近端梯度法的特例。
- ▶ ISTA 是基础近端梯度法; FISTA 加入外推, 在凸复合问题中可达到 $\mathcal{O}(1/k^2)$ 。
- ▶ 加速可能带来震荡, 实践中常用 restart 稳定迭代。

课后思考

如果一个问题的 g 很自然, 但 prox_g 很难算, 近端梯度法还适合作为首选算法吗?